

Final_Chatbot_Petty

June 14, 2026

1 Final Chatbot

1.0.1 NYTimes Crossword Hint Generator

Ryan Petty

2 Model 1:

```
[ ]: !pip install transformers datasets accelerate
```

```
[ ]: import re
import numpy as np
from tensorflow import keras
from keras.layers import Input, LSTM, Dense, Embedding, Concatenate, AdditiveAttention
from keras.models import Model, load_model
import os

import csv
import random

os.environ["CUDA_VISIBLE_DEVICES"] = "-1"

# Define start and end tokens
START_TOKEN = "START"
END_TOKEN = "END"

print("Loading and formatting combined datasets (NYT + WordNet): ")
pairs = []

# Load NYT Data
try:
    with open("nyt_crossword_clues_kaggle.txt", 'r', encoding='utf-8') as f:
        reader = csv.DictReader(f)
        for row in reader:
            answer = str(row.get('answer', '')).strip()
```

```

raw_clue = str(row.get('clue', '')).strip()

if answer and raw_clue:
    length_constraint = str(len(answer))
    clean_clue = re.sub(r'\s*(\d+(?:,\d+)*\)$', '', raw_clue).
strip()

    input_string = f"{answer} length {length_constraint}"
    target_string = clean_clue
    pairs.append((input_string, target_string))
except FileNotFoundError:
    print("Warning: nyt_crossword_clues_kaggle.txt not found. Skipping.")

# Load Princeton WordNet Data
try:
    with open("wordnet_core_word_senses_princeton.txt", 'r', encoding='utf-8')
as f:
    for line in f:
        if ']' in line:
            parts = line.split(']')
            word = parts[-2].replace('[', '').strip()
            definition = parts[-1].strip()

            if word and definition:
                length_constraint = str(len(word))
                input_string = f"{word} length {length_constraint}"
                target_string = definition
                pairs.append((input_string, target_string))
except FileNotFoundError:
    print("Warning: wordnet_core_word_senses_princeton.txt not found. Skipping.
")

# Shuffle the combined dataset
random.seed(42)
random.shuffle(pairs)

print(f"Successfully loaded and shuffled {len(pairs)} combined pairs.")

input_docs, target_docs = [], []
input_tokens, target_tokens = set(), set()

# Increased data from 400 to 10000 pairs
for input_line, target_line in pairs[:10000]:
    input_docs.append(input_line)
    # Use explicit START and END tokens
    target_line = START_TOKEN + ' ' + target_line + ' ' + END_TOKEN
    target_docs.append(target_line)

```

```

    for token in re.findall(r"[\w']+|[\^\s\w]", input_line):
        input_tokens.add(token)
    for token in target_line.split():
        target_tokens.add(token)

# Add START and END tokens explicitly
target_tokens.add(START_TOKEN)
target_tokens.add(END_TOKEN)

# Add a padding token (usually index 0)
PAD_TOKEN = "<PAD>"
input_tokens.add(PAD_TOKEN)
target_tokens.add(PAD_TOKEN)

input_tokens = sorted(list(input_tokens))
target_tokens = sorted(list(target_tokens))
num_encoder_tokens = len(input_tokens)
num_decoder_tokens = len(target_tokens)

print(f"Input vocabulary size: {num_encoder_tokens}")
print(f"Target vocabulary size: {num_decoder_tokens}")

input_features_dict = {token: i for i, token in enumerate(input_tokens)}
target_features_dict = {token: i for i, token in enumerate(target_tokens)}
reverse_input_features_dict = {i: token for token, i in input_features_dict.
    ↪items()}
reverse_target_features_dict = {i: token for token, i in target_features_dict.
    ↪items()}

max_encoder_seq_length = max(len(re.findall(r"[\w']+|[\^\s\w]", doc)) for doc in
    ↪input_docs)
max_decoder_seq_length = max(len(doc.split()) for doc in target_docs)

print(f"Max encoder sequence length: {max_encoder_seq_length}")
print(f"Max decoder sequence length: {max_decoder_seq_length}")

# Now creating integer sequences instead of one-hot arrays
encoder_input_data = np.zeros((len(input_docs), max_encoder_seq_length),
    ↪dtype='float32')
decoder_input_data = np.zeros((len(input_docs), max_decoder_seq_length),
    ↪dtype='float32')
# Target data is still one-hot for categorical crossentropy, or we can use
    ↪sparse categorical crossentropy
# Let's switch to sparse categorical crossentropy to save memory

```

```

decoder_target_data = np.zeros((len(input_docs), max_decoder_seq_length, 1),
    dtype='float32')

print("Vectorizing sequences: ")
for line, (input_doc, target_doc) in enumerate(zip(input_docs, target_docs)):
    for t, token in enumerate(re.findall(r"[\w']+|^[^s\w]", input_doc)):
        if token in input_features_dict:
            encoder_input_data[line, t] = input_features_dict[token]

    target_tokens_list = target_doc.split()
    for t, token in enumerate(target_tokens_list):
        if token in target_features_dict:
            decoder_input_data[line, t] = target_features_dict[token]
        if t > 0:
            decoder_target_data[line, t - 1, 0] =
target_features_dict[token]

# Model setup
dimensionality = 128
embedding_dim = 100 # Define embedding dimension
batch_size = 128
epochs = 20

print("Building model with Embeddings and Attention: ")

# Encoder
encoder_inputs = Input(shape=(None,))
encoder_embedding = Embedding(num_encoder_tokens, embedding_dim)(encoder_inputs)
encoder_lstm = LSTM(dimensionality, return_sequences=True, return_state=True,
    activation='tanh', recurrent_activation='sigmoid', unroll=True)
encoder_outputs, state_h, state_c = encoder_lstm(encoder_embedding)
encoder_states = [state_h, state_c]

# Decoder
decoder_inputs = Input(shape=(None,))
decoder_embedding_layer = Embedding(num_decoder_tokens, embedding_dim)
decoder_embedding = decoder_embedding_layer(decoder_inputs)

decoder_lstm = LSTM(dimensionality, return_sequences=True, return_state=True,
    activation='tanh', recurrent_activation='sigmoid', unroll=True)
decoder_outputs, _, _ = decoder_lstm(decoder_embedding,
    initial_state=encoder_states)

# Attention Layer
attention_layer = AdditiveAttention(name='attention_layer')
attention_out = attention_layer([decoder_outputs, encoder_outputs])

```

```

# Concatenate attention output and decoder LSTM output
decoder_concat_input = Concatenate(axis=-1,␣
    ↪name='concat_layer')([decoder_outputs, attention_out])

# Dense layer
decoder_dense = Dense(num_decoder_tokens, activation='softmax')
decoder_outputs = decoder_dense(decoder_concat_input)

# Compile model using sparse_categorical_crossentropy
training_model = Model([encoder_inputs, decoder_inputs], decoder_outputs)
training_model.compile(optimizer='rmsprop',␣
    ↪loss='sparse_categorical_crossentropy', metrics=['accuracy'])

print("Training model: ")
training_model.fit([encoder_input_data, decoder_input_data],␣
    ↪decoder_target_data,
                    batch_size=batch_size, epochs=epochs, validation_split=0.2)
print("Training completed.")
# Save weights instead of full model to avoid custom layer loading issues␣
    ↪sometimes
training_model.save_weights("training_model.weights.h5")

print("Setting up inference models: ")
# Encoder inference model
encoder_model = Model(encoder_inputs, [encoder_outputs] + encoder_states)

# Decoder inference model
decoder_state_input_h = Input(shape=(dimensionality,))
decoder_state_input_c = Input(shape=(dimensionality,))
decoder_states_inputs = [decoder_state_input_h, decoder_state_input_c]
encoder_outputs_input = Input(shape=(None, dimensionality))

dec_emb2 = decoder_embedding_layer(decoder_inputs)
decoder_outputs2, state_h2, state_c2 = decoder_lstm(dec_emb2,␣
    ↪initial_state=decoder_states_inputs)
decoder_states2 = [state_h2, state_c2]

# Attention in inference
attention_out2 = attention_layer([decoder_outputs2, encoder_outputs_input])
decoder_concat_input2 = Concatenate(axis=-1)([decoder_outputs2, attention_out2])

decoder_outputs2 = decoder_dense(decoder_concat_input2)

decoder_model = Model(
    [decoder_inputs] + decoder_states_inputs + [encoder_outputs_input],
    [decoder_outputs2] + decoder_states2
)

```

```

def decode_response(test_input):
    enc_out, h, c = encoder_model.predict(test_input)
    states_value = [h, c]
    target_seq = np.zeros((1, 1))

    target_seq[0, 0] = target_features_dict[START_TOKEN]

    decoded_sentence = ''
    while True:
        output_tokens, h, c = decoder_model.predict([target_seq] + states_value)
        ↪+ [enc_out])

        def sampleT(probabilities, temperature=1.0):
            predictions = np.asarray(probabilities).astype('float64')
            predictions = np.log(predictions + 1e-10) / temperature
            exp_preds = np.exp(predictions)
            predictions = exp_preds / np.sum(exp_preds)
            return np.random.choice(len(predictions), p=predictions)

        sampled_index = sampleT(output_tokens[0, -1, :], temperature=0.7)
        sampled_token = reverse_target_features_dict.get(sampled_index, "")

        if sampled_token == END_TOKEN:
            break
        if sampled_token != START_TOKEN and sampled_token != PAD_TOKEN:
            decoded_sentence += ' ' + sampled_token

        if len(decoded_sentence.split()) > max_decoder_seq_length:
            break

        target_seq = np.zeros((1, 1))
        target_seq[0, 0] = sampled_index
        states_value = [h, c]

    return decoded_sentence.strip()

class ChatBot:
    exit_commands = ("quit", "exit", "bye", "goodbye", "stop")

    def start_chat(self):
        print("ChatBot ready! Type a message or 'exit' to quit.")
        user_input = input("You: ")
        while not self.should_exit(user_input):
            response = self.generate_response(user_input)
            print("Bot:", response)
            user_input = input("You: ")

```

```

        print("Bot: Goodbye!")

    def generate_response(self, user_input):
        user_matrix = self.sentence_to_matrix(user_input)
        return decode_response(user_matrix)

    def sentence_to_matrix(self, sentence):
        tokens = re.findall(r"[\w']+|[\^\s\w]", sentence)
        matrix = np.zeros((1, max_encoder_seq_length), dtype='float32')
        for t, token in enumerate(tokens):
            if token in input_features_dict and t < max_encoder_seq_length:
                matrix[0, t] = input_features_dict[token]
        return matrix

    def should_exit(self, reply):
        return any(exit_cmd in reply.lower() for exit_cmd in self.exit_commands)

def please_chat():
    chatbot = ChatBot()
    chatbot.start_chat()

#if __name__ == "__main__":
# Uncomment to train and chat
# please_chat()

```

Loading and formatting combined datasets (NYT + WordNet):

Warning: nyt_crossword_clues_kaggle.txt not found. Skipping.

Successfully loaded and shuffled 4521 combined pairs.

Input vocabulary size: 3292

Target vocabulary size: 6678

Max encoder sequence length: 5

Max decoder sequence length: 17

Vectorizing sequences:

Building model with Embeddings and Attention:

Training model:

Epoch 1/20

29/29 93s 3s/step -

accuracy: 0.7383 - loss: 5.0805 - val_accuracy: 0.7600 - val_loss: 1.9250

Epoch 2/20

29/29 132s 3s/step -

accuracy: 0.7659 - loss: 1.7596 - val_accuracy: 0.7726 - val_loss: 1.7359

Epoch 3/20

29/29 76s 2s/step -

accuracy: 0.7957 - loss: 1.6224 - val_accuracy: 0.8187 - val_loss: 1.6585

Epoch 4/20

29/29 69s 2s/step -

accuracy: 0.8231 - loss: 1.5484 - val_accuracy: 0.8187 - val_loss: 1.6121

Epoch 5/20
 29/29 68s 2s/step -
 accuracy: 0.8243 - loss: 1.5144 - val_accuracy: 0.8187 - val_loss: 1.6012
 Epoch 6/20
 29/29 83s 2s/step -
 accuracy: 0.8243 - loss: 1.4991 - val_accuracy: 0.8187 - val_loss: 1.6025
 Epoch 7/20
 29/29 87s 3s/step -
 accuracy: 0.8243 - loss: 1.4907 - val_accuracy: 0.8187 - val_loss: 1.6022
 Epoch 8/20
 29/29 75s 3s/step -
 accuracy: 0.8243 - loss: 1.4846 - val_accuracy: 0.8187 - val_loss: 1.6054
 Epoch 9/20
 29/29 79s 2s/step -
 accuracy: 0.8243 - loss: 1.4804 - val_accuracy: 0.8187 - val_loss: 1.6090
 Epoch 10/20
 29/29 70s 2s/step -
 accuracy: 0.8243 - loss: 1.4761 - val_accuracy: 0.8188 - val_loss: 1.6165
 Epoch 11/20
 29/29 83s 2s/step -
 accuracy: 0.8243 - loss: 1.4729 - val_accuracy: 0.8188 - val_loss: 1.6168
 Epoch 12/20
 29/29 81s 2s/step -
 accuracy: 0.8243 - loss: 1.4697 - val_accuracy: 0.8189 - val_loss: 1.6211
 Epoch 13/20
 29/29 82s 2s/step -
 accuracy: 0.8243 - loss: 1.4664 - val_accuracy: 0.8189 - val_loss: 1.6255
 Epoch 14/20
 29/29 82s 2s/step -
 accuracy: 0.8246 - loss: 1.4633 - val_accuracy: 0.8187 - val_loss: 1.6262
 Epoch 15/20
 29/29 82s 2s/step -
 accuracy: 0.8245 - loss: 1.4604 - val_accuracy: 0.8189 - val_loss: 1.6272
 Epoch 16/20
 29/29 72s 2s/step -
 accuracy: 0.8247 - loss: 1.4573 - val_accuracy: 0.8189 - val_loss: 1.6305
 Epoch 17/20
 29/29 72s 2s/step -
 accuracy: 0.8246 - loss: 1.4544 - val_accuracy: 0.8189 - val_loss: 1.6343
 Epoch 18/20
 29/29 81s 2s/step -
 accuracy: 0.8247 - loss: 1.4519 - val_accuracy: 0.8189 - val_loss: 1.6345
 Epoch 19/20
 29/29 70s 2s/step -
 accuracy: 0.8246 - loss: 1.4486 - val_accuracy: 0.8189 - val_loss: 1.6398
 Epoch 20/20
 29/29 81s 2s/step -
 accuracy: 0.8247 - loss: 1.4458 - val_accuracy: 0.8189 - val_loss: 1.6377

Training completed.
Setting up inference models:

2.0.1 Output:

Even after some tweaks we're not getting great results

ChatBot ready! Type a message or 'exit' to quit.

You: Acquistive chap, as we see it

Bot: not in point

You: Foreign letter

Bot: rock of

You: Grant papers

Bot: tight, of

You: A little house might ... be this? (4)

Bot: auction connected

3 Model 2: Using Pre-Trained Model

Attempting to use t5-small model

```
[ ]: import pandas as pd
import csv
import re
import torch
from datasets import Dataset
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, Seq2SeqTrainer, Seq2SeqTrainingArguments, DataCollatorForSeq2Seq

print("Loading and formatting data for T5...")
pairs = []

# We will use the NYT dataset to teach it the "crossword voice"
with open("nytimes_crossword_clues_kaggle.txt", 'r', encoding='utf-8') as f:
    reader = csv.DictReader(f)
    for row in reader:
        answer = str(row.get('answer', '')).strip()
        raw_clue = str(row.get('clue', '')).strip()

        if answer and raw_clue:
            length = str(len(answer))
            clean_clue = re.sub(r'\s*(\d+(?:,\d+)*\)$', '', raw_clue).strip()
```

```

        # T5 relies heavily on task prefixes to understand what to do
        input_string = f"generate crossword clue: {answer} (length_
↪{length})"
        pairs.append({"input_text": input_string, "target_text":_
↪clean_clue})

# 15,000 rows is the sweet spot for a fast Colab run that still learns
df = pd.DataFrame(pairs).sample(15000, random_state=42)
dataset = Dataset.from_pandas(df)
dataset = dataset.train_test_split(test_size=0.1)

print("Loading pre-trained T5-small model and tokenizer...")
model_checkpoint = "t5-small"
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
model = AutoModelForSeq2SeqLM.from_pretrained(model_checkpoint)

def preprocess_function(examples):
    model_inputs = tokenizer(examples["input_text"], max_length=32,_
↪truncation=True)
    labels = tokenizer(text_target=examples["target_text"], max_length=32,_
↪truncation=True)
    model_inputs["labels"] = labels["input_ids"]
    return model_inputs

print("Tokenizing datasets...")
tokenized_datasets = dataset.map(preprocess_function, batched=True)

# Training Arguments optimized for Colab T4
args = Seq2SeqTrainingArguments(
    output_dir="./t5-clue-generator",
    eval_strategy="epoch",
    learning_rate=3e-4,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    weight_decay=0.01,
    save_total_limit=1,
    num_train_epochs=4,
    predict_with_generate=True,
    fp16=True,
    logging_steps=50,
)

data_collator = DataCollatorForSeq2Seq(tokenizer, model=model)

trainer = Seq2SeqTrainer(
    model=model,

```

```

    args=args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["test"],
    data_collator=data_collator
)

print("Starting training...")
trainer.train()

```

Loading and formatting data for T5...

Loading pre-trained T5-small model and tokenizer...

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93:

UserWarning:

The secret `HF\_TOKEN` does not exist in your Colab secrets.

To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as secret in your Google Colab and restart your session.

You will be able to reuse this secret in all of your notebooks.

Please note that authentication is recommended but still optional to access public models or datasets.

warnings.warn(

config.json: 0.00B [00:00, ?B/s]

tokenizer_config.json: 0.00B [00:00, ?B/s]

spiece.model: 0%| | 0.00/792k [00:00<?, ?B/s]

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

tokenizer.json: 0.00B [00:00, ?B/s]

model.safetensors: 0%| | 0.00/242M [00:00<?, ?B/s]

Loading weights: 0%| | 0/131 [00:00<?, ?it/s]

generation_config.json: 0%| | 0.00/147 [00:00<?, ?B/s]

Tokenizing datasets...

Map: 0%| | 0/13500 [00:00<?, ? examples/s]

Map: 0%| | 0/1500 [00:00<?, ? examples/s]

Starting training...

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775:

UserWarning: 'pin_memory' argument is set as true but no accelerator is found,

then device pinned memory won't be used.

```
super().__init__(loader)
```

<IPython.core.display.HTML object>

```
Writing model shards: 0%|          | 0/1 [00:00<?, ?it/s]
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775:
```

```
UserWarning: 'pin_memory' argument is set as true but no accelerator is found,  
then device pinned memory won't be used.
```

```
super().__init__(loader)
```

```
Writing model shards: 0%|          | 0/1 [00:00<?, ?it/s]
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775:
```

```
UserWarning: 'pin_memory' argument is set as true but no accelerator is found,  
then device pinned memory won't be used.
```

```
super().__init__(loader)
```

```
Writing model shards: 0%|          | 0/1 [00:00<?, ?it/s]
```

```
Writing model shards: 0%|          | 0/1 [00:00<?, ?it/s]
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775:
```

```
UserWarning: 'pin_memory' argument is set as true but no accelerator is found,  
then device pinned memory won't be used.
```

```
super().__init__(loader)
```

```
[ ]: TrainOutput(global_step=1688, training_loss=4.989982324753893,  
metrics={'train_runtime': 5359.243, 'train_samples_per_second': 10.076,  
'train_steps_per_second': 0.315, 'total_flos': 245112461328384.0, 'train_loss':  
4.989982324753893, 'epoch': 4.0})
```

```
[ ]: # The Chat Interface  
def generate_clue(answer):  
    length = len(answer.replace(" ", ""))  
    input_text = f"generate crossword clue: {answer} (length {length})"  
  
    # Move inputs to GPU  
    device = "cuda" if torch.cuda.is_available() else "cpu"  
    print("debug: using {}".format(device))  
    inputs = tokenizer(input_text, return_tensors="pt").to(device)  
    model.to(device)  
  
    # Generate using beam search for better linguistic quality  
    outputs = model.generate(  
        inputs.input_ids,  
        max_length=32,  
        num_beams=4,  
        early_stopping=True  
    )
```

```

    clue = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return clue

# Try some novel words it hasn't seen!
print(generate_clue("PYTHON"))
print(generate_clue("JUPYTER"))
print(generate_clue("MACHINE LEARNING"))
# and words it might have seen
print(generate_clue("COVETOUS"))
print(generate_clue("DROOPY"))
print(generate_clue("EXAM"))
print(generate_clue("BANANAS"))
print(generate_clue("ADDUCED"))
print(generate_clue("EPISTLE"))
# from wordnet
print(generate_clue("ABLE"))
print(generate_clue("ABSENT"))
print(generate_clue("ABSOLUTE"))

```

```

debug: using cpu
One's a sailor's sailor
debug: using cpu
A sailor's sailor's sailor
debug: using cpu
What's a sailor's sailor's sailor
debug: using cpu
A sailor's sailor's sailor's sailor
debug: using cpu
Reportedly, he's a sailor's sailor
debug: using cpu
A sailor's sailor's sailor
debug: using cpu
A sailor's sailor's sailor
debug: using cpu
Reportedly snared to be snared
debug: using cpu
Irregular sailor's sailor's sailor
debug: using cpu
Having a sailor, a sailor is able to be able
debug: using cpu
A sailor's a sailor's sailor
debug: using cpu
Disagreement of a sailor's sailor

```

While the text generation is repetitive, it seems like it is actually learning the patterns of speech found in NYT crossword puzzles now. “Reportedly...” is a clue for a “sounds like...” answer, and “One’s a...” phrases often act as the key to understanding a theme where several answers fit the

description provided.

Output:

One's a sailor's sailor
A sailor's sailor's sailor
What's a sailor's sailor's sailor
A sailor's sailor's sailor's sailor
Reportedly, he's a sailor's sailor
A sailor's sailor's sailor
A sailor's sailor's sailor
Reportedly snared to be snared
Irregular sailor's sailor's sailor
Having a sailor, a sailor is able to be able
A sailor's a sailor's sailor
Disagreement of a sailor's sailor

4 Model 3

Adding in saving off of the model, lowering the learning rate, adding in another data source, add nucleus sampling and repetition penalty to help speech sound better.

Now that it is working better and not being repetitive, training on the whole dataset.

```
[ ]: import pandas as pd
import csv
import re
import torch
import os
from google.colab import drive
from datasets import Dataset
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, Seq2SeqTrainer, \
    Seq2SeqTrainingArguments, DataCollatorForSeq2Seq

# Mount Google Drive to save model permanently
print("Mounting Google Drive...")
drive.mount('/content/drive')

# Define the save path on Drive
drive_model_path = "/content/drive/MyDrive/NLP_Crossword_Project/t5_final_model"

# Check if the model is already trained and saved
if os.path.exists(drive_model_path):
    print(f"Saved model found at {drive_model_path}!")
    print("Skipping training and loading model directly into memory...")
    tokenizer = AutoTokenizer.from_pretrained(drive_model_path)
    model = AutoModelForSeq2SeqLM.from_pretrained(drive_model_path)
```

```

else:
    print("No saved model found. Beginning data prep and training...")

    # --- DATA PREP ---
    import random

    pairs = []

    # Load NYT Data
    try:
        with open("nytimes_crossword_clues_kaggle.txt", 'r', encoding='utf-8')
↪ as f:
            reader = csv.DictReader(f)
            for row in reader:
                answer = str(row.get('answer', '')).strip()
                raw_clue = str(row.get('clue', '')).strip()
                if answer and raw_clue:
                    length = str(len(answer))
                    clean_clue = re.sub(r'\s*\(\d+(?:,\d+)*\)$', '', raw_clue).
↪ strip()
                    pairs.append({"input_text": f"generate crossword clue:
↪ {answer} (length {length})", "target_text": clean_clue})
            except FileNotFoundError:
                print("WARNING: upload nyt_crossword_clues_kaggle.txt to Colab!")

    # Load WordNet Data
    try:
        with open("wordnet_core_word_senses_princeton.txt", 'r',
↪ encoding='utf-8') as f:
            for line in f:
                if ']' in line:
                    parts = line.split(']')
                    word = parts[-2].replace('[', '').strip().upper()
                    definition = parts[-1].strip()
                    if word and definition:
                        length = str(len(word))
                        pairs.append({"input_text": f"generate crossword clue:
↪ {word} (length {length})", "target_text": definition})
            except FileNotFoundError:
                print("WARNING: upload wordnet_core_word_senses_princeton.txt to Colab!
↪ ")

    # Load ChatGPT Synthetic Data
    try:

```

```

    with open("chatgpt_synthetic_hints_updated.csv", 'r', encoding='utf-8')
↪as f:
        reader = csv.DictReader(f)
        for row in reader:
            answer = str(row.get('word', '')).strip().upper()
            raw_clue = str(row.get('hint', '')).strip()
            if answer and raw_clue:
                length = str(len(answer))
                clean_clue = re.sub(r'\s*(\d+(?:,\d+)*\)$', '', raw_clue).
↪strip()
                pairs.append({"input_text": f"generate crossword clue:
↪{answer} (length {length})", "target_text": clean_clue})
        except FileNotFoundError:
            print("WARNING: upload chatgpt_synthetic_hints_updated.csv to Colab!")

    print(f"Loaded an unrestricted total of {len(pairs)} pairs.")

    # Shuffle the combined list
    random.shuffle(pairs)

    # Convert to Dataset
    df = pd.DataFrame(pairs)
    dataset = Dataset.from_pandas(df).train_test_split(test_size=0.1)

    print(f"Proceeding with full dataset training on {len(df)} rows.")

    # --- MODEL SETUP ---
    model_checkpoint = "t5-base"
    # model_checkpoint = "t5-small"
    tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
    model = AutoModelForSeq2SeqLM.from_pretrained(model_checkpoint)

    def preprocess_function(examples):
        model_inputs = tokenizer(examples["input_text"], max_length=32,
↪truncation=True)
        labels = tokenizer(text_target=examples["target_text"], max_length=32,
↪truncation=True)
        model_inputs["labels"] = labels["input_ids"]
        return model_inputs

    tokenized_datasets = dataset.map(preprocess_function, batched=True)

    # --- TRAINING ARGUMENTS ---
    args = Seq2SeqTrainingArguments(
        output_dir="/content/drive/MyDrive/NLP_Crossword_Project/checkpoints",
        eval_strategy="epoch",
        learning_rate=1e-4, # Lowered

```



```

        per_device_train_batch_size=16,
        per_device_eval_batch_size=16,
        weight_decay=0.01,
        save_total_limit=2,
        num_train_epochs=4,
        predict_with_generate=True,
        fp16=False,
        optim="adamw_torch", # Default PyTorch optimizer
        logging_steps=100,
    )

    data_collator = DataCollatorForSeq2Seq(tokenizer, model=model)

    trainer = Seq2SeqTrainer(
        model=model,
        args=args,
        train_dataset=tokenized_datasets["train"],
        eval_dataset=tokenized_datasets["test"],
        data_collator=data_collator,
    )

    # --- TRAIN AND SAVE ---
    print("Starting training...")
    trainer.train()

    print("Training complete! Saving final model to Google Drive...")
    trainer.save_model(drive_model_path)
    tokenizer.save_pretrained(drive_model_path)
    print("Model successfully saved to Drive!")

# The Inference/Generation Interface With Anti-Looping safeguards
def generate_clue(answer):
    length = len(answer.replace(" ", ""))
    input_text = f"generate crossword clue: {answer} (length {length})"

    device = "cuda" if torch.cuda.is_available() else "cpu"

    inputs = tokenizer(input_text, return_tensors="pt").to(device)
    model.to(device)

    outputs = model.generate(
        inputs.input_ids,
        max_length=32,
        do_sample=True,
        top_k=50,
        top_p=0.95,

```

```

        temperature=0.8,
        repetition_penalty=2.5    # Prevents repeating words
    )

    clue = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return clue

print("\n--- Model is ready for testing! ---")
print(f"PYTHON: {generate_clue('PYTHON')}")
print(f"MACHINE LEARNING: {generate_clue('MACHINE LEARNING')}")

```

Mounting Google Drive...

Mounted at /content/drive

Saved model found at

/content/drive/MyDrive/NLP_Crossword_Project/t5_final_model!

Skipping training and loading model directly into memory...

Loading weights: 0%| | 0/131 [00:00<?, ?it/s]

--- Model is ready for testing! ---

PYTHON: One of two aforementioned that's in danger

MACHINE LEARNING: Playing with a man, at that time: it's in the spirit of something to be held

```

[ ]: # Try some words it hasn't seen
print(f"PYTHON: {generate_clue('PYTHON')}")
print(f"JUPYTER: {generate_clue('JUPYTER')}")
print(f"MACHINE LEARNING: {generate_clue('MACHINE LEARNING')}")
print(f"NATURAL LANGUAGE PROCESSING: {generate_clue('NATURAL LANGUAGE_
↳PROCESSING')}")
# and words it has seen
print(f"COVETOUS: {generate_clue('COVETOUS')}")
print(f"DROOPY: {generate_clue('DROOPY')}")
print(f"EXAM: {generate_clue('EXAM')}")
print(f"BANANAS: {generate_clue('BANANAS')}")
print(f"ADDUCED: {generate_clue('ADDUCED')}")
print(f"EPISTLE: {generate_clue('EPISTLE')}")
# words from wordnet
print(f"ABLE: {generate_clue('ABLE')}")
print(f"ABSENT: {generate_clue('ABSENT')}")
print(f"ABSOLUTE: {generate_clue('ABSOLUTE')}")

```

PYTHON: Small bit of a cat's tongue

JUPYTER: One with less patience at first about a person of skill

MACHINE LEARNING: Taking action with the French, one's in charge of education

NATURAL LANGUAGE PROCESSING: Confrontation with the subject of a study to provide guidance?

COVETOUS: No doubt a party has no end
DROOPY: Plot snatching large prize
EXAM: Time to remove a few of the men
BANANAS: Sick to do with a long time in North American city
ADDUCED: Suggested a new job at the time
EPISTLE: Girl is a part of the sea
ABLE: Insufficient material, one with little water
ABSENT: Deaf or right to get free
ABSOLUTE: Defame one's hysterical, and let off?

4.0.1 Evaluating Model 3

I will use the **ROUGE** metric. It is the standard evaluation metric for sequence-to-sequence tasks like summarization and text generation with T5. It measures the overlap of n-grams between the model's generated clues and the actual target clues.

Results are as expected - fairly low scores. Because the chatbot is trained on NYT crossword clues, which are relatively short, its output is quite small. This, combined with the inherent complexity and oddities found in crossword clues, leads to a low number of matches in the ROUGE evaluation. A much better evaluation is reading responses, seeing if they have the feel of crossword clues and they make relative sense compared to the answer. Of course, many NYT crossword clues don't make much sense to me in the first place, so if this was a production model we would want somebody much more experienced to analyze the clues.

```
[ ]: !pip install evaluate rouge_score
```

```
[ ]: import evaluate
import numpy as np

# Load the ROUGE metric
rouge = evaluate.load("rouge")

# We will evaluate on a small sample of the test set (100 examples) to keep it
↳ fast
sample_test_dataset = tokenized_datasets["test"].select(range(100))

def compute_metrics(eval_preds):
    preds, labels = eval_preds
    if isinstance(preds, tuple):
        preds = preds[0]

    # Replace -100 in the labels
    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)

    # Decode predictions and references
    decoded_preds = tokenizer.batch_decode(preds, skip_special_tokens=True)
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)
```

```

    # Compute ROUGE
    result = rouge.compute(predictions=decoded_preds,
↪references=decoded_labels, use_stemmer=True)
    return result

# Attach the metric function to the trainer
trainer.compute_metrics = compute_metrics

print("Evaluating on a test sample...")
eval_results = trainer.evaluate(eval_dataset=sample_test_dataset)

print("\n--- Evaluation Results ---")
for key, value in eval_results.items():
    if "rouge" in key:
        print(f"{key}: {value:.4f}")

```

Downloading builder script: 0.00B [00:00, ?B/s]

Evaluating on a test sample...

<IPython.core.display.HTML object>

```

--- Evaluation Results ---
eval_rouge1: 0.0452
eval_rouge2: 0.0024
eval_rougeL: 0.0427
eval_rougeLsum: 0.0428

```

4.0.2 Interactive Chat Loop

```

[ ]: def start_clue_generator():
    print("T5-Base Crossword Clue Generator ready! Type a word or 'exit' to
↪quit.")
    while True:
        user_input = input("You (Enter Answer): ")
        if user_input.lower() in ['quit', 'exit', 'bye', 'goodbye', 'stop']:
            print("Bot: Goodbye!")
            break
        if not user_input.strip():
            continue

    # Format to uppercase for consistency with training data
    word = user_input.strip().upper()
    response = generate_clue(word)
    print(f"Bot (Clue for {word}): {response}\n")

```

```
start_clue_generator()
```

T5-Base Crossword Clue Generator ready! Type a word or 'exit' to quit.

You (Enter Answer): Graph

Bot (Clue for GRAPH): Item enveloping one's face?

```
-----
KeyboardInterrupt                                Traceback (most recent call last)
/tmp/ipykernel_4076/288317660.py in <cell line: 0>()
    15
    16
--> 17 start_clue_generator()

/tmp/ipykernel_4076/288317660.py in start_clue_generator()
      2     print("T5-Base Crossword Clue Generator ready! Type a word or 'exit'
↳to quit.")
      3     while True:
----> 4         user_input = input("You (Enter Answer): ")
      5         if user_input.lower() in ['quit', 'exit', 'bye', 'goodbye',
↳'stop']:
      6             print("Bot: Goodbye!")

/usr/local/lib/python3.12/dist-packages/ipykernel/kernelbase.py in
↳_raw_input(self, prompt)
    1175         "raw_input was called, but this frontend does not
↳support input requests."
    1176     )
-> 1177     return self._input_request(
    1178         str(prompt),
    1179         self._parent_ident["shell"],

/usr/local/lib/python3.12/dist-packages/ipykernel/kernelbase.py in
↳_input_request(self, prompt, ident, parent, password)
    1217     except KeyboardInterrupt:
    1218         # re-raise KeyboardInterrupt, to truncate traceback
-> 1219         raise KeyboardInterrupt("Interrupted by user") from Non
    1220     except Exception:
    1221         self.log.warning("Invalid Message:", exc_info=True)

KeyboardInterrupt: Interrupted by user
```